

Projet SIO-Shoes : site « e-commerce » avec Symfony

Partie 7 : ajouter, modifier et supprimer des produits

1. Ajout des produits.

Nous allons créer une entité « Product » qui contiendra les propriétés « name », « description » et « price ». Nous ajouterons d'autres propriétés par la suite.

Commencez par la commande : `symfony console make:entity Product`

```
PS C:\Symfony\e-commerce> symfony console make:entity Product
created: src/Entity/Product.php
created: src/Repository/ProductRepository.php

Entity generated! Now let's add some fields!
You can always add more fields later manually or by re-running this command.

New property name (press <return> to stop adding fields):
> name

Field type (enter ? to see all types) [string]:
>

Field length [255]:
>

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Product.php

Add another property? Enter the property name (or press <return> to stop adding fields):
> description

Field type (enter ? to see all types) [string]:
> text

Can this field be null in the database (nullable) (yes/no) [no]:
> yes

updated: src/Entity/Product.php

Add another property? Enter the property name (or press <return> to stop adding fields):
> price
```

Pour le champ « price », nous allons choisir le type « decimal » :

```

Field type (enter ? to see all types) [string]:
> decimal

Precision (total number of digits stored: 100.00 would be 5) [10]:
>

Scale (number of decimals to store: 100.00 would be 2) [0]:
> 2

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Product.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>

Success!

Next: When you're ready, create a migration with symfony.exe console make:migration

```

Remarque : il est possible d'afficher la liste des types. Référez-vous à la documentation pour plus d'informations.

```

Field type (enter ? to see all types) [string]:
> ?

Main Types
* string or ascii_string
* text
* boolean
* integer or smallint or bigint
* float

Relationships/Associations
* relation a wizard will help you build the relation
* ManyToOne
* OneToMany
* ManyToMany
* OneToOne

Array/Object Types
* array or simple_array
* json
* object
* binary
* blob

Date/Time Types
* datetime or datetime_immutable
* datetimetz or datetimetz_immutable
* date or date_immutable
* time or time_immutable
* dateinterval

Other Types
* enum
* decimal
* guid

```

2. Préparez la migration pour créer la table « Product » dans la BDD :

```
symfony console make:migration
```

Puis exécutez la migration avec doctrine :

```
symfony console doctrine:migrations:migrate
```

3. Ajout des sous-catégories à un produit

Dans symfony, il est possible de modifier une entité en relançant la commande `symfony console make:entity` en mettant en paramètre le nom d'une entité existante.

Modifiez l'entité « Product » avec la commande `symfony console make:entity Product`

Ajoutez un champ « subCategories » de type ManyToMany (un produit possède 0 à N sous-catégories, une sous-catégorie contient 0 à N produits) :

```
PS C:\Symfony\e-commerce> symfony console make:entity Product
Your entity already exists! So let's add some new fields!

New property name (press <return> to stop adding fields):
> subCategories

Field type (enter ? to see all types) [string]:
> ManyToMany

What class should this entity be related to?:
> SubCategory

Do you want to add a new property to SubCategory so that you can access/update Product objects from it - e.g.
$subCategory->getProducts()? (yes/no) [yes]:
> yes

A new property will also be added to the SubCategory class so that you can access the related Product objects
from it.

New field name inside SubCategory [products]:
>

updated: src/Entity/Product.php
updated: src/Entity/SubCategory.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>

Success!

Next: When you're ready, create a migration with symfony.exe console make:migration
```

4. Préparez et exécutez la migration pour mettre à jour la BDD :

```
symfony console make:migration
```

```
symfony console doctrine:migrations:migrate
```

Vous pouvez voir le résultat dans phpMyAdmin avec la table « product_sub_category » qui a bien été créée :



#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
☐	1 product_id	int(11)			Non	Aucun(e)			Modifier Supprimer Plus
☐	2 sub_category_id	int(11)			Non	Aucun(e)			Modifier Supprimer Plus

5. Créez le CRUD pour l'entité « Product » :

symfony console make:crud

```
The class name of the entity to create CRUD (e.g. OrangePopsicle):
> Product

Choose a name for your controller class (e.g. ProductController) [ProductController]:
>

Do you want to generate PHPUnit tests? [Experimental] (yes/no) [no]:
>

created: src/Controller/ProductController.php
created: src/Form/ProductForm.php
created: templates/product/_delete_form.html.twig
created: templates/product/_form.html.twig
created: templates/product/edit.html.twig
created: templates/product/index.html.twig
created: templates/product/new.html.twig
created: templates/product/show.html.twig

Success!

Next: Check your new CRUD by going to /product/
PS C:\Users\islero\source\repos\Symfony\e-commerce> |
```

6. Rendre le nom de produit unique.

Modifiez le fichier « src\Entity\Product.php » en commençant par importer la bibliothèque « UniqueEntity » : `use Symfony\Bridge\Doctrine\Validator\Constraints\UniqueEntity;`

Ajoutez le code suivant juste avant la classe « Product » :

```
#[UniqueEntity(
    fields: ['name'],
    message: 'Ce nom de produit existe déjà',
    errorPath: 'name',
)]
```

Modifiez le fichier « templates\product_form.html.twig » pour afficher le message en cas d'erreur :

```
{{ form_start(form) }}

<label for="">Nom du produit : </label>
{{ form_widget(form.name, {'attr':{'class':'form form-control','placeholder':"nom"}}) }}
{{ form_errors(form.name) }}
```

TRAVAIL A FAIRE POUR LES ETUDIANTS :

- Réserver la gestion des produits aux utilisateurs ayant le rôle « ROLE_EDITOR ».
- Ajouter la gestion des produits dans la navbar (uniquement pour les administrateurs et les éditeurs). Rappel : les administrateurs sont aussi des éditeurs.
- Mettre en forme toutes les vues qui concernent les produits (avec Bootstrap). Remarque : pour afficher seulement une partie du texte d'un champ, vous pouvez utiliser la fonction « slice() ». Par exemple, dans le fichier « templates\product\index.html.twig », vous pouvez limiter l'affichage de la description avec le code suivant :

```
<td> {{ product.description|slice(0,100) }} ...</td>
```
- Ajouter des messages flash lors de l'ajout la modification et la suppression d'un produit.